

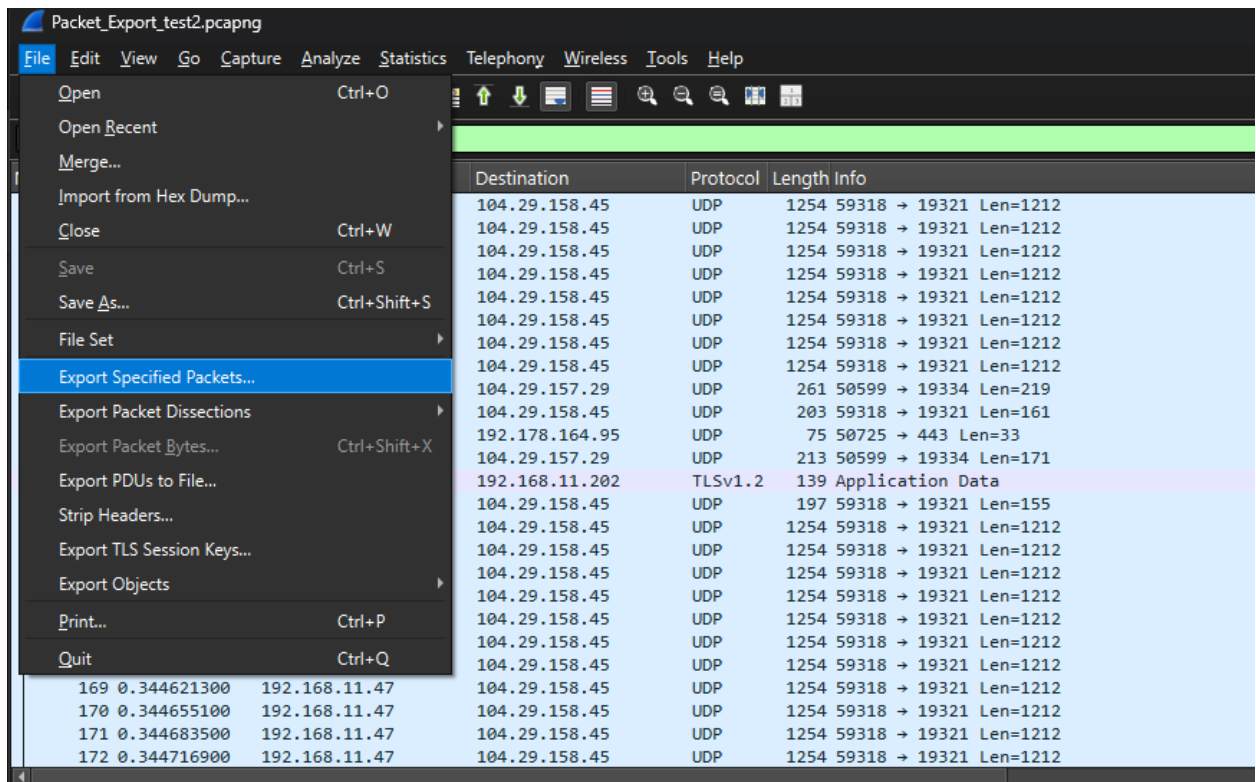
Wireshark Certification WCA-101

Notes from 25 Apr 2026

1.0 Utilize key features of Wireshark (10%)

- Open, save, and close files, and export specified packets.
For exporting we needed to set a filter “ip.src==192.168.0.0/16” to capture the packets that we wanted to. Just selecting them, did nothing.

This was a useful site: <https://wiki.wireshark.org/DisplayFilters>



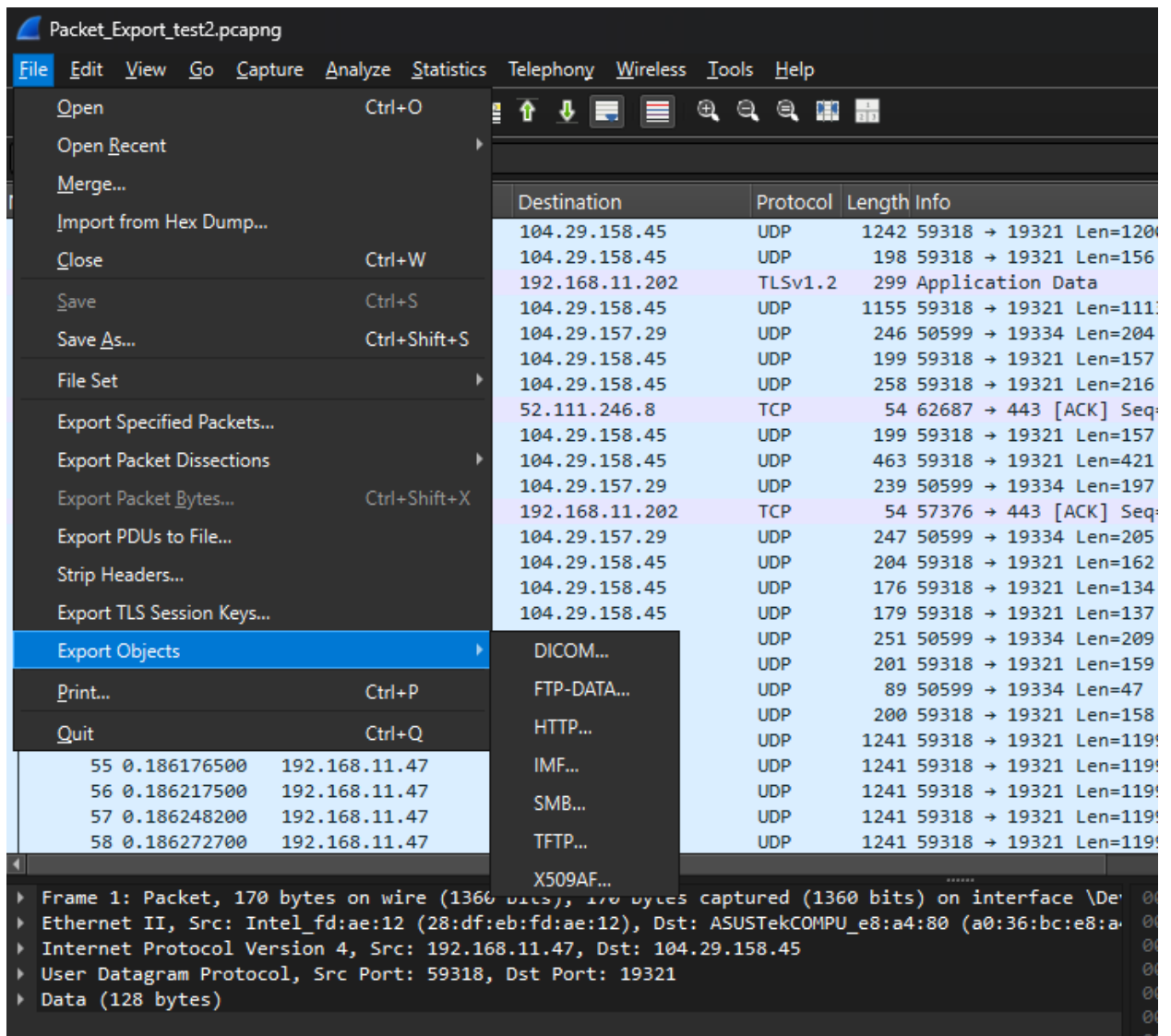
- Describe the difference between different capture file formats, especially pcap and pcapng.

Review: <https://wiki.wireshark.org/Development/LibpcapFileFormat>

<https://wiki.wireshark.org/libpcap> <https://medium.com/@pagaudiomonte/pcapng-vs-pcap-whats-the-diff-6670ca2273d3>

Major differences is that .pcap is more legacy (and what we pull from Linux boxes). .pcapng is the default now for captures in wireshark.

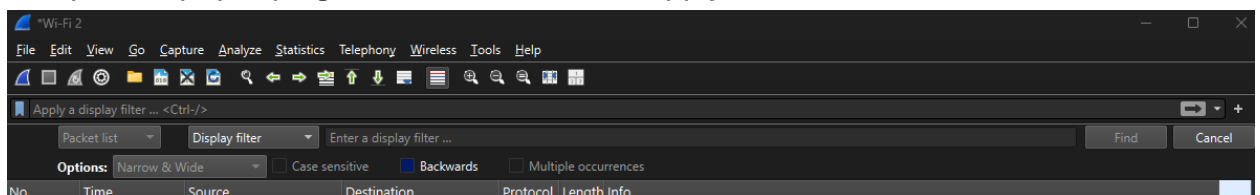
- Export objects from packet captures.



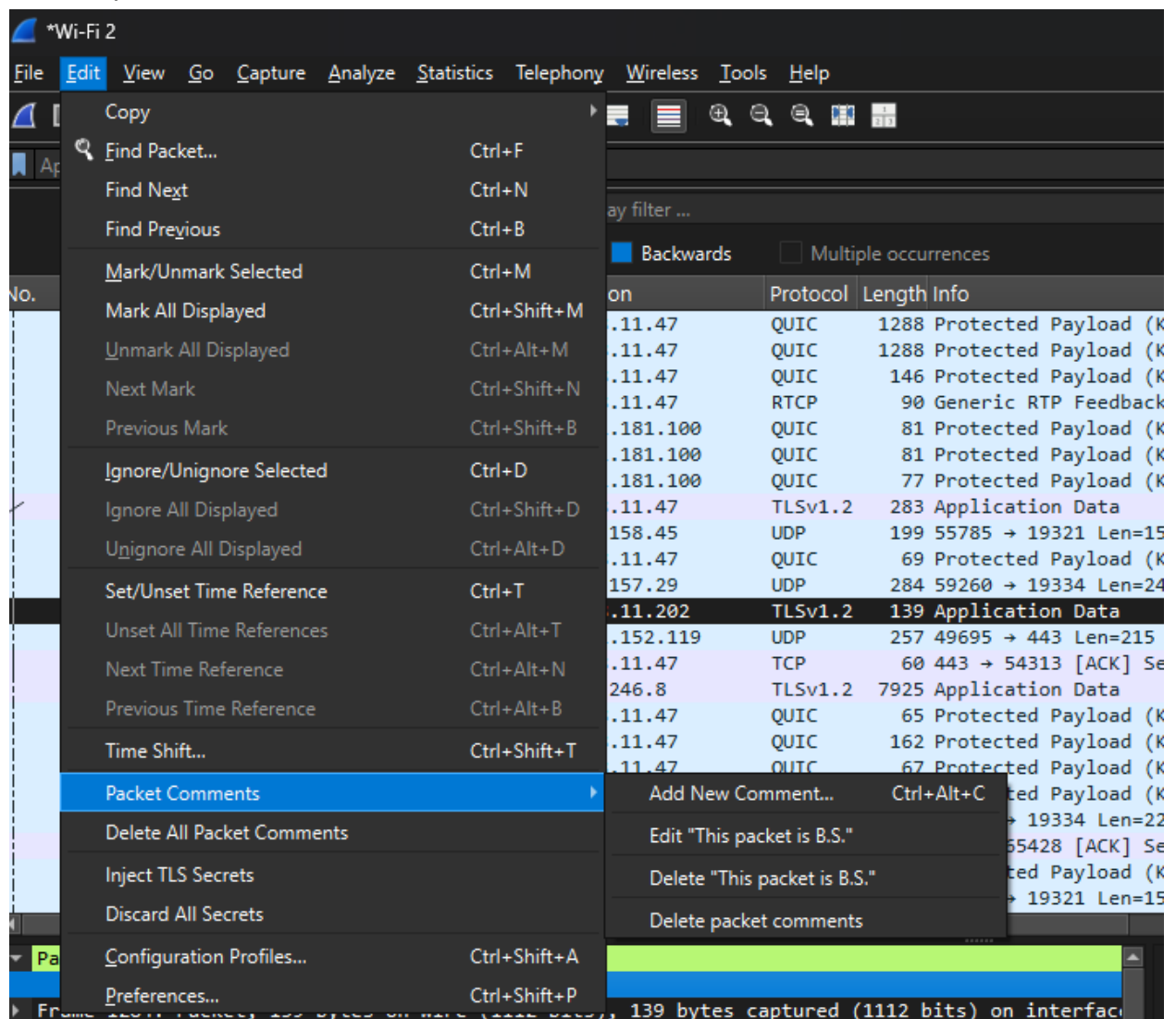
Read about the different file types: DICOM, FTP-DATA, HTTP, IMF, SMB, TFTP, X509AF

- Use 'Find Packet' to locate packets of interest.
Edit -> Find Packet or "Ctrl + F"

Find just keeps jumping to the next, it does not apply a filter to the screen.

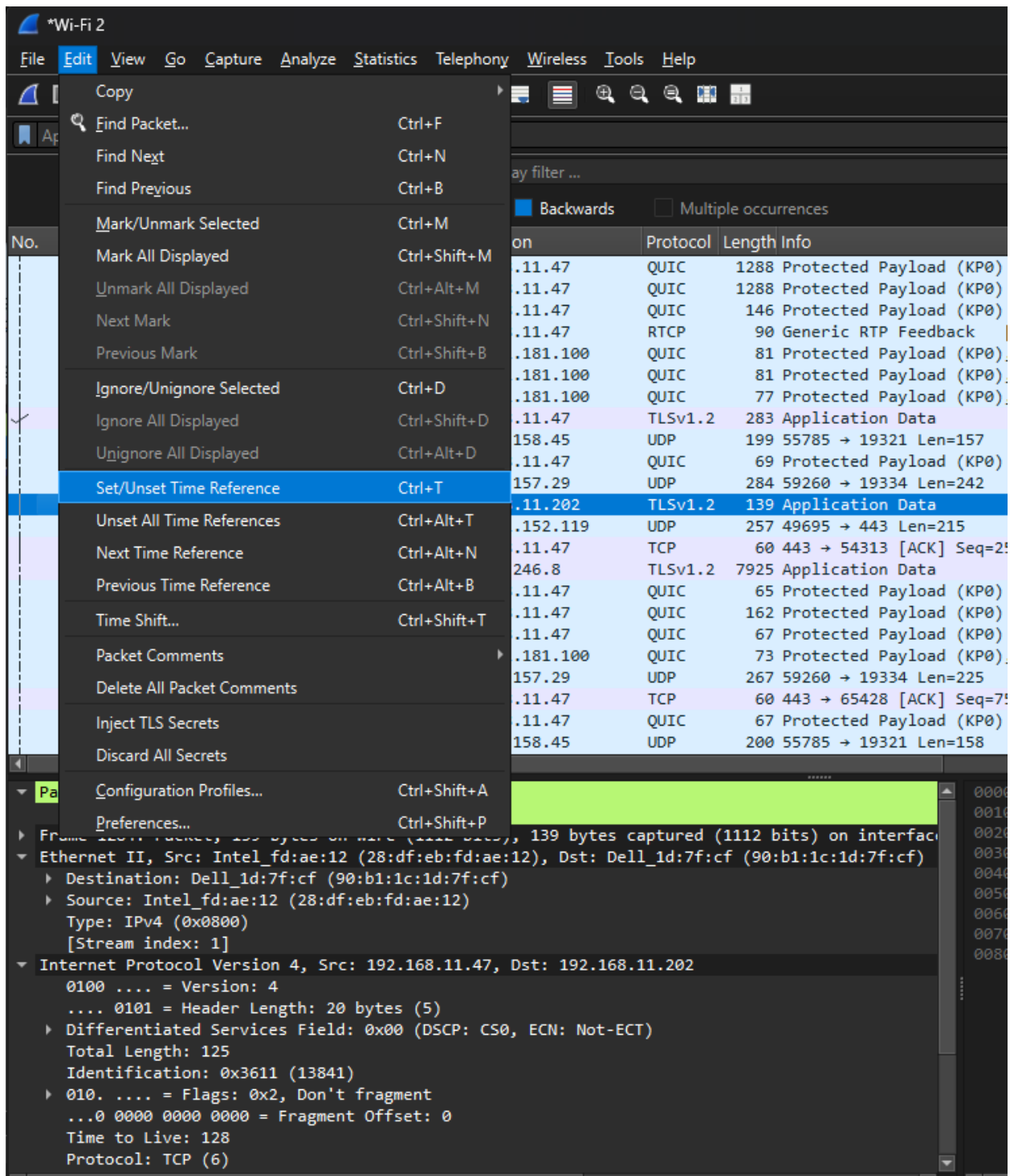


- Use the packet and file comments feature.

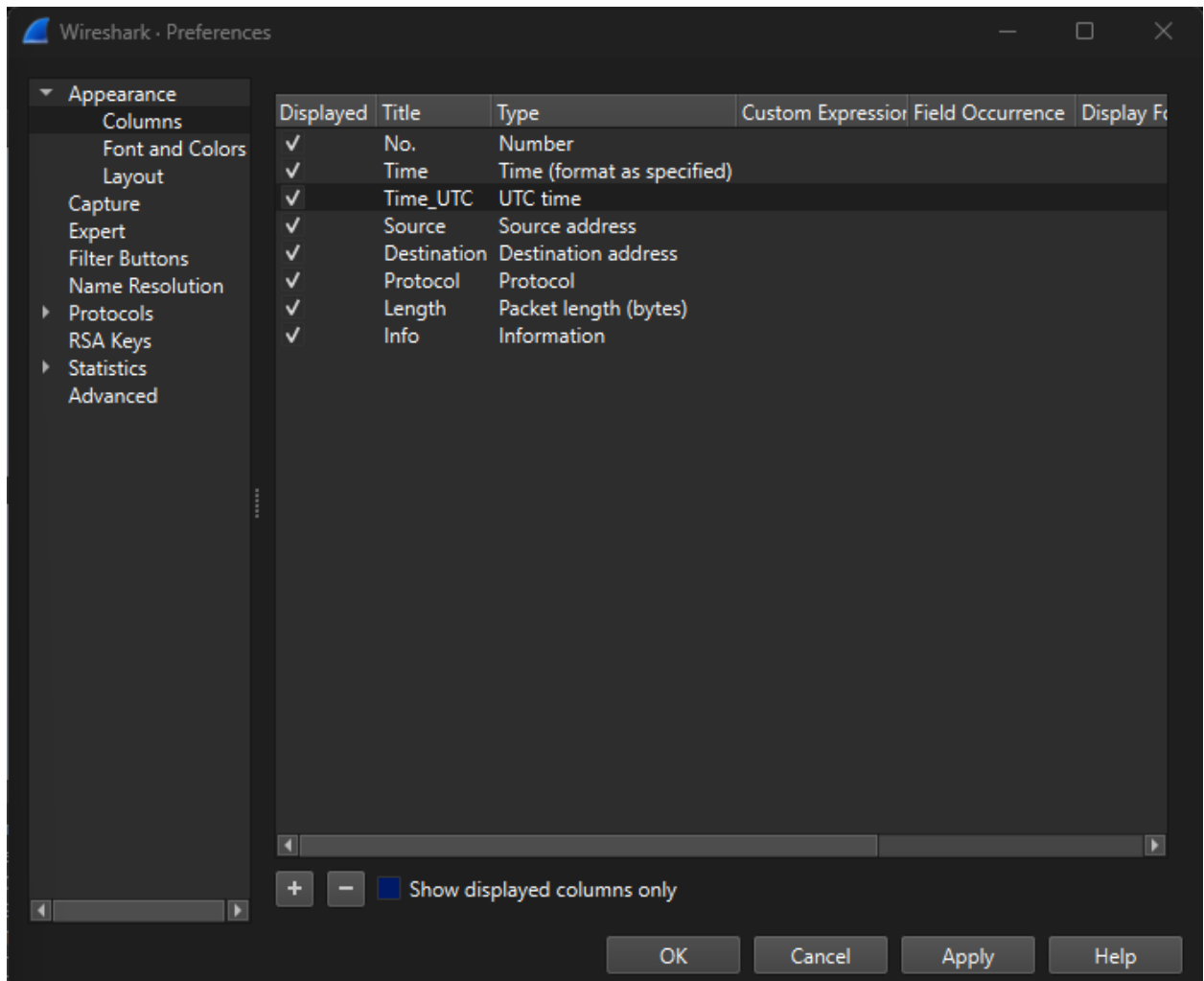


- Set/Unset time reference.
From where you set this, time starts at "0", like a zero day, or zero time from this packet etc.

Once you have more than one (1), you can jump between them with Next and Previous time references.



- Apply different time formats in a capture.
Edit -> Preferences -> Columns
Can apply any number of time formats
You can move it in the preferences tool, or out in the actual view of packets.



- Describe how Wireshark applies different name resolution options.
It is unchecked by default, but we can set name resolution in edit -> preferences -> Name Resolution -> Resolve network (IP) addresses.
- This is saved with the file in .pcapng files.
- Configure name resolution.
Same as above, you can also set a column with name resolved instead of the default IP address.
- Use 'Decode as' feature.
Applies to the packet that you are focused on.
Analyze -> Decode As OR you can right click on a packet -> "Decode As"

- Use 'Capture File Properties' to identify key information about the capture.

Statistics -> Capture File Properties

The screenshot shows the 'Wireshark - Capture File Properties - Wi-Fi 2' window. It displays various details about the capture file, organized into sections: File, Time, Capture, Interfaces, and Statistics.

File

- Name: C:\Users\het_t\AppData\Local\Temp\wireshark_Wi-Fi 2QUZ8N3.pcapng
- Length: 8832 kB
- Hash (SHA256): 1a45b6ba7ffa942df9f720010fe59bf21a557c19a5e0b13bf5457a3f2328ad64
- Hash (SHA1): 4010f6ac779ba169ddc71889d99cd0c0e53d67af
- Format: Wireshark/... - pcapng
- Encapsulation: Ethernet

Time

- First packet: 2026-04-25 15:58:28
- Last packet: 2026-04-25 15:58:44
- Elapsed: 00:00:15

Capture

- Hardware: Intel(R) Core(TM) i7-10870H CPU @ 2.20GHz (with SSE4.2)
- OS: 64-bit Windows 11 (25H2), build 26200
- Application: Dumpcap (Wireshark) 4.6.4 (v4.6.4-0-g93282876538d)

Interfaces

Interface	Dropped packets	Capture filter	Link type	Packet size limit (snaplen)
Wi-Fi 2	0 (0.0%)	none	Ethernet	262144 bytes

Statistics

Measurement	Captured	Displayed	Marked
Packets	10089	10089 (100.0%)	—
Time span, s	15.857	15.857	—
Average pps	636.2	636.2	—
Average packet size, B	842	842	—
Bytes	8495117	8495117 (100.0%)	0
Average bytes/s	535 k	535 k	—
Average bits/s	4285 k	4285 k	—

At the bottom of the window, there are buttons for 'Refresh', 'Edit Comments', 'Close', 'Copy To Clipboard', and 'Help'.

- Use 'Protocol Hierarchy' to identify key protocols in a capture.

Statistics -> Protocol Hierarchy

Aggregates all the traffic up into percentages of overall traffic.

Wireshark - Protocol Hierarchy Statistics - Wi-Fi 2

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDU's
Frame	100.0	10089	100.0	8495117	4285 k	0	0	0	10089
Ethernet	100.0	10089	1.7	143068	72 k	0	0	0	10089
Internet Protocol Version 4	99.7	10054	2.4	201088	101 k	0	0	0	10054
User Datagram Protocol	88.1	8890	0.8	71120	35 k	0	0	0	8890
Simple Service Discovery Protocol	0.2	22	0.1	9748	4917	22	9748	4917	22
Real-time Transport Control Protocol	3.7	373	0.2	17928	9044	223	6984	3523	398
Malformed Packet	1.5	150	0.0	0	0	150	0	0	150
QUIC IETF	11.6	1174	7.8	659647	332 k	1174	650850	328 k	1202
Multicast Domain Name System	0.0	4	0.0	492	248	4	492	248	4
Domain Name System	0.0	4	0.0	288	145	4	288	145	4
Data	72.5	7313	78.7	6689043	3374 k	7313	6689043	3374 k	7313
Transmission Control Protocol	11.5	1162	0.3	24356	12 k	692	14956	7545	1162
Transport Layer Security	3.9	391	7.7	655906	330 k	391	654593	330 k	392
Domain Name System	0.8	78	0.1	6577	3318	78	6577	3318	78
Data	0.0	1	0.0	1	0	1	1	0	1
Internet Group Management Protocol	0.0	2	0.0	32	16	2	32	16	2
Address Resolution Protocol	0.3	35	0.0	980	494	35	980	494	35

- Use 'Conversations' to identify key communications in a capture.

Statistics -> Conversations

Shows overall frames between systems

Shows data moving between systems

Wireshark - Conversations - Wi-Fi 2

Conversation Settings

- ☒ Name resolution
- ☒ Absolute start time
- ☒ Display raw data
- ☒ Limit to display filter

Copy

Follow Stream...

Graph...

I/O Graphs

Protocol

- ☐ Bluetooth
- ☐ BPV7
- ☐ DCCP
- ☐ DNP 3.0
- ☒ Ethernet
- ☐ FC
- ☐ FDDI
- ☐ IEEE 802.11
- ☐ IEEE 802.15.4
- ☐ ILNP
- ☒ IPv4
- ☒ IPv6
- ☐ IPX
- ☐ iVTA

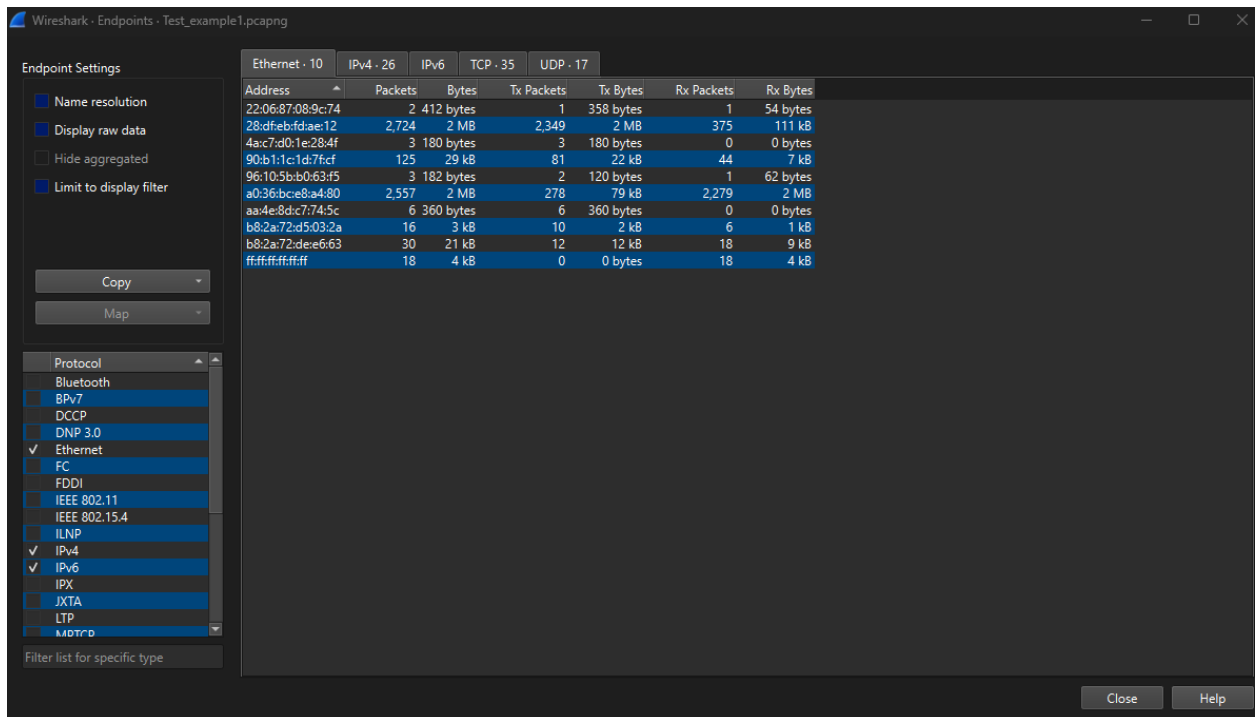
Filter list for specific type

Address A	Address B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start
28:dfefbdae:12	01:00:5e:00:00:16	2	108 bytes	5	2	108 bytes	0	0 bytes	3.723088300
28:dfefbdae:12	90:b1:1c:20:74:2a	6	1 kB	9	3	636 bytes	3	515 bytes	6.423187400
28:dfefbdae:12	b8:2a:72:d5:03:2a	13	2 kB	3	6	1 kB	7	1 kB	0.554170200
28:dfefbdae:12	b8:2a:72:de:e6:63	70	53 kB	1	42	23 kB	28	30 kB	0.199818000
28:dfefbdae:12	c8:1f:66:d1:b3:37	6	1 kB	10	3	636 bytes	3	515 bytes	9.531676800
4a:c7:d0:1e:28:4f	ff:ff:ff:ff:ff:ff	27	2 kB	8	27	2 kB	0	0 bytes	4.995083800
90:b1:1c:1d:7f:cf	28:dfefbdae:12	257	50 kB	2	169	37 kB	88	13 kB	0.293999600
96:10:5b:b0:63:f5	28:dfefbdae:12	2	114 bytes	11	1	60 bytes	1	54 bytes	9.903277100
a0:36:bce8:a4:80	01:00:5e:7f:ff:fa	22	11 kB	6	22	11 kB	0	0 bytes	3.767802600
a0:36:bce8:a4:80	28:dfefbdae:12	9,672	8 MB	0	1,385	655 kB	8,287	8 MB	0.000000000
aa:4e:8dc7:74:5c	ff:ff:ff:ff:ff:ff	8	480 bytes	4	8	480 bytes	0	0 bytes	1.512700400
b0:c5:54:25:2d:45	01:00:5e:00:00:fb	4	660 bytes	7	4	660 bytes	0	0 bytes	4.586166600

Close Help

- Use 'Endpoints' to analyze host traffic in a capture.

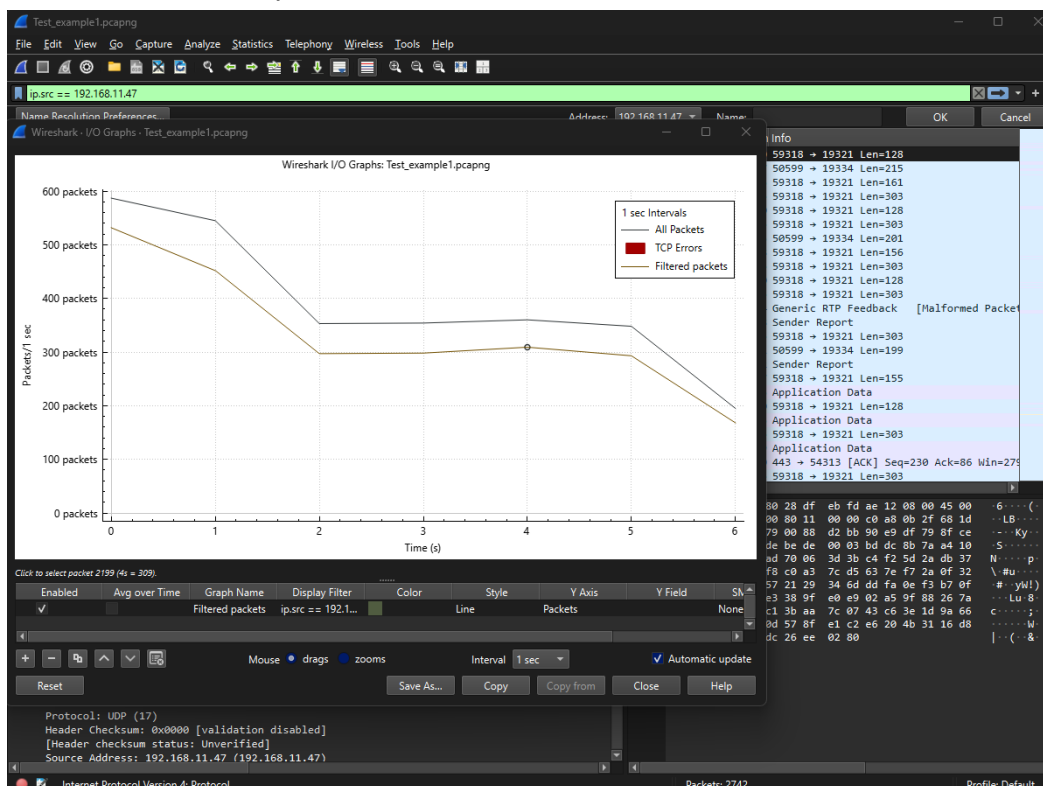
Statistics -> Endpoints



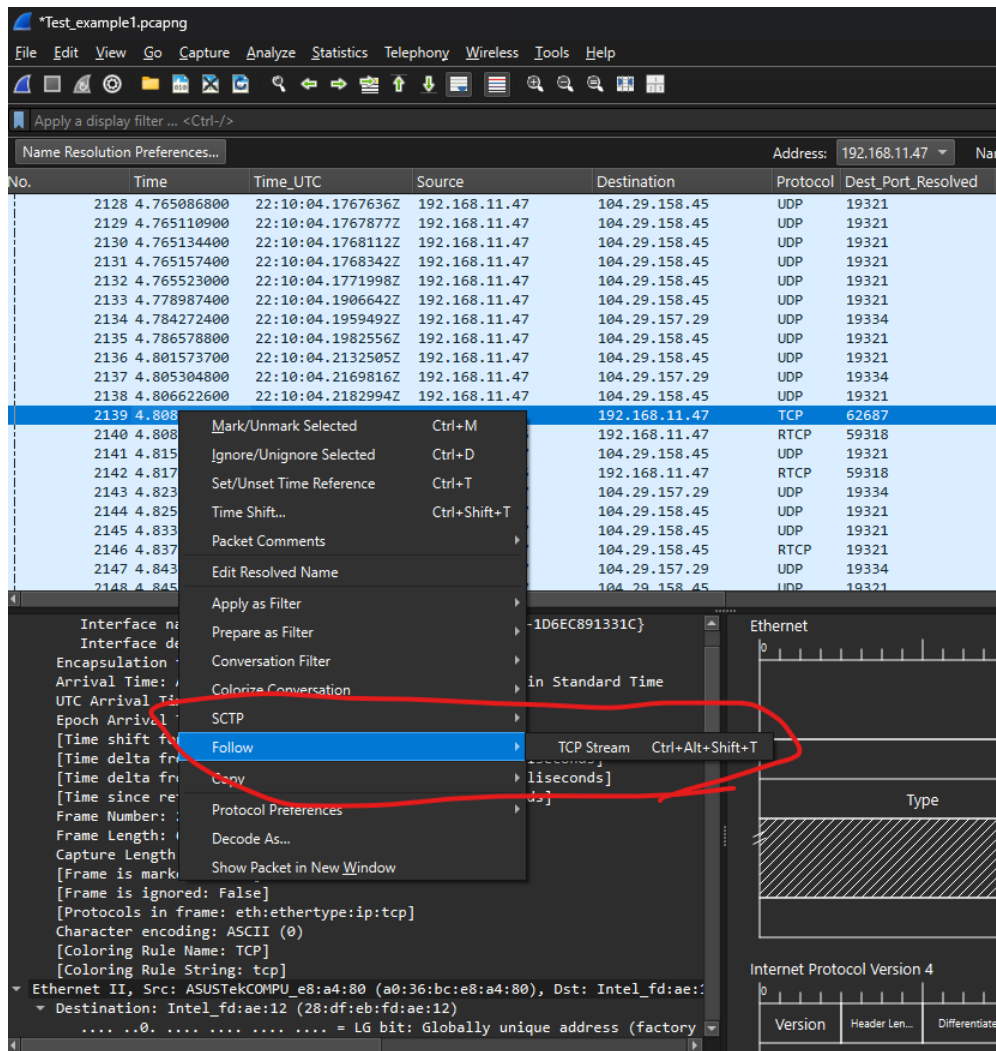
- Create and interpret an I/O graph that shows packets/s or bits/s for a given display filter.

Set a filter “ip.src == 192.168.11.27”

Statistics -> I/O Graphs



- Distinguish between actual bytes captured and fields generated by the Wireshark Metadata is seen in [], in the frame information. It is not captured from the line.
- dissectors. (shown in [])
Synthetic metadata in the packet details that is not actually captured at part of the packet. It's interpreted from the binary data.
- Use 'Follow TCP/UDP' stream.
Right click on a TCP Stream -> Follow TCP/UDP stream and then click "Follow"



Future study for Section 1:

- https://gaia.cs.umass.edu/kurose_ross/online_lectures.htm

Notes from 2 May 2026

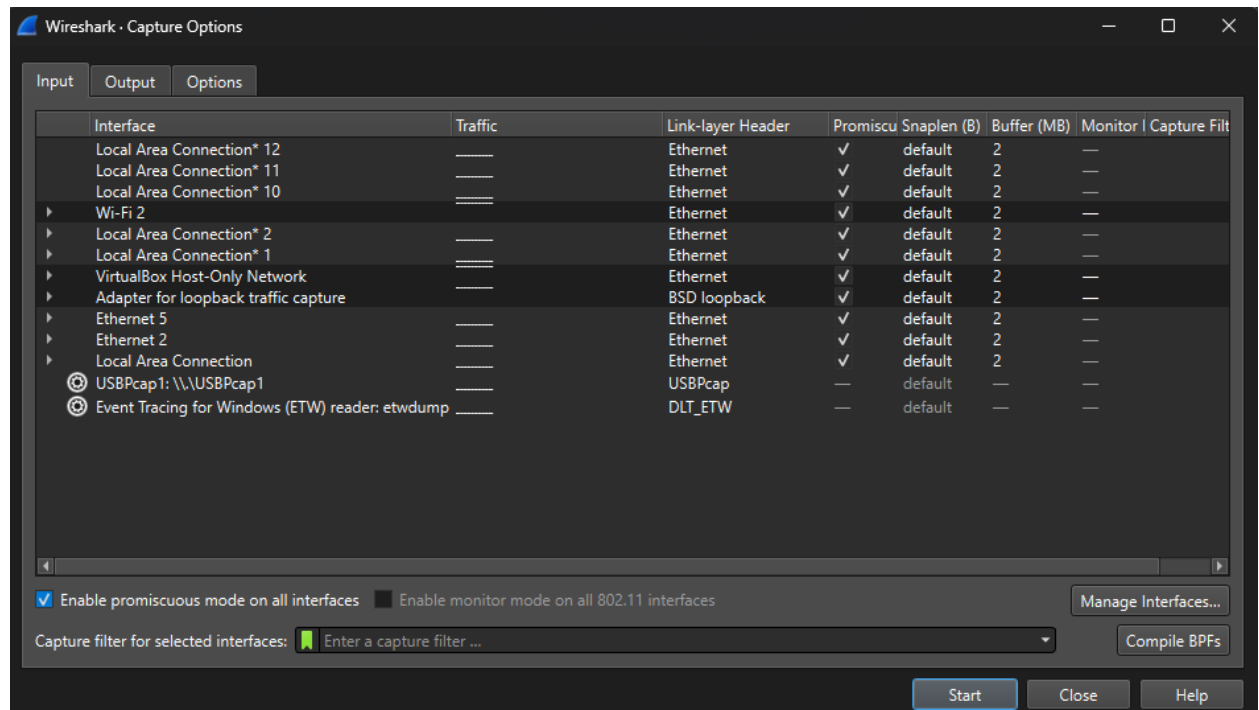
2.0 Utilize different methods of Capturing Traffic (10%)

- Compare and contrast the benefits of different methods used for traffic capture. (Direct on Endpoint, Network Tap, Infrastructure Device, Port Mirror, Multi-Point Capture)
 - Direct on Endpoint is captured on the system you want to listen from.
 - Network Tap – Physical hardware that is between the switch and the device.
 - Can be undetected in the case of electromagnetic systems.
 - Infrastructure devices – Physical hardware between systems.
 - Multi-Point Capture - is the collection of many interfaces at the same time.

- Select the appropriate interface to capture traffic in Wireshark.

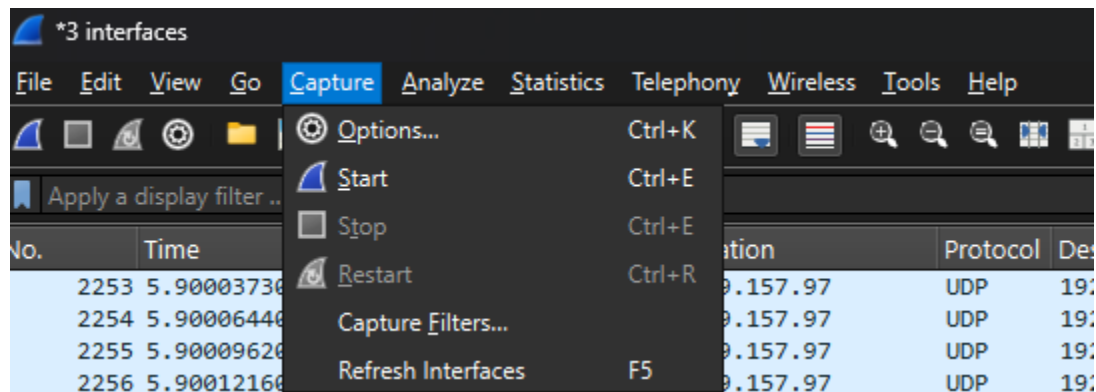
Capture -> options

Use “CTRL” to select multiple before capture.



- Start/Stop/Restart Capture in Wireshark.

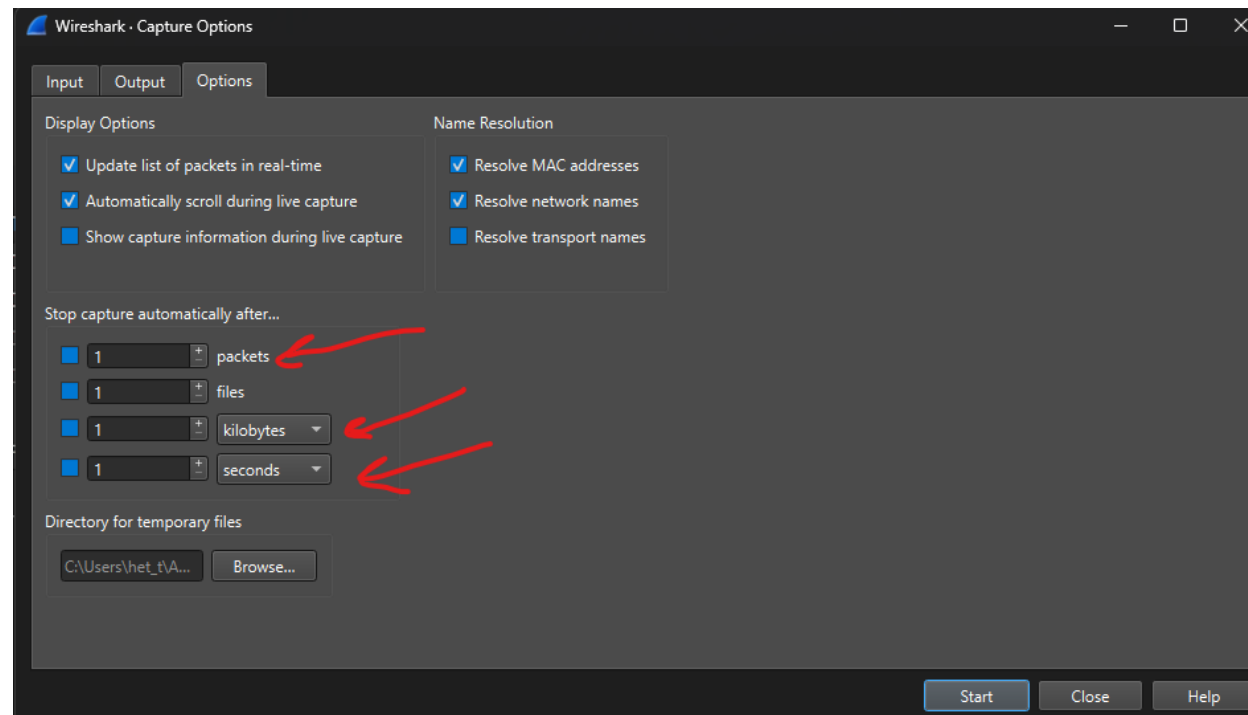
Capture -> Option you want to select



- Limit capture by file size, packets, or duration.

- Wireshark

Capture -> Options -> Tab Options

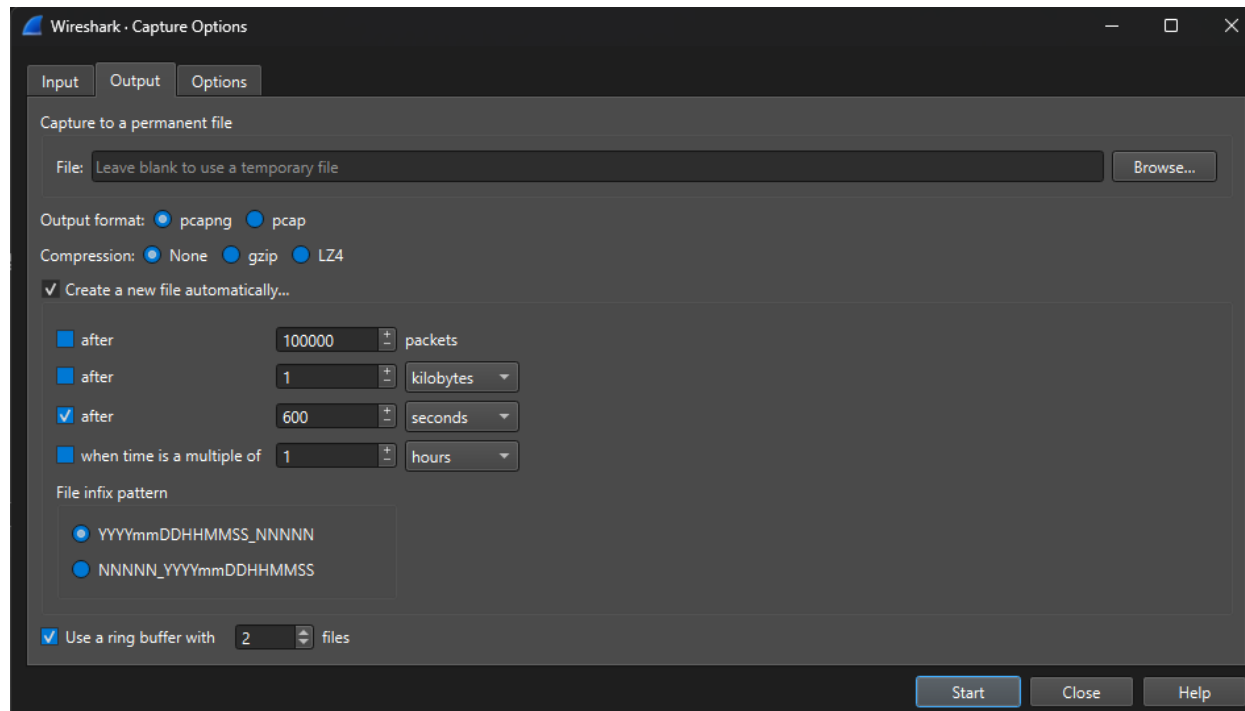


- Tcpcap

- By file size '-C'
- By number of packets '-c' or count
- By duration of time '-G'
 - USE `tcpdump -h` to reference these.

- Implement a Ring Buffer.

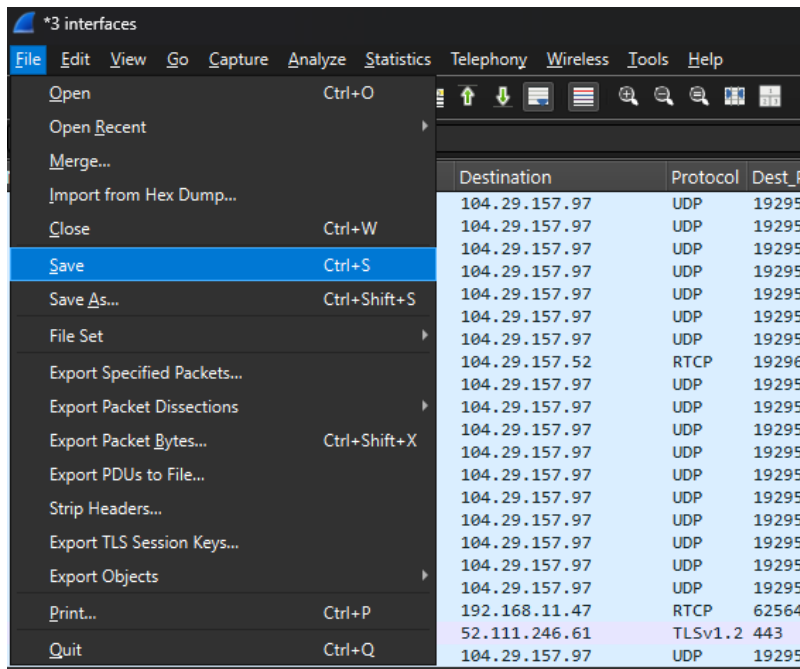
- Wireshark's ring buffer is a capture mode for long-term monitoring that saves data across multiple, rotating files to prevent filling up disk space.
- Steps – Capture -> Options -> Output Tab



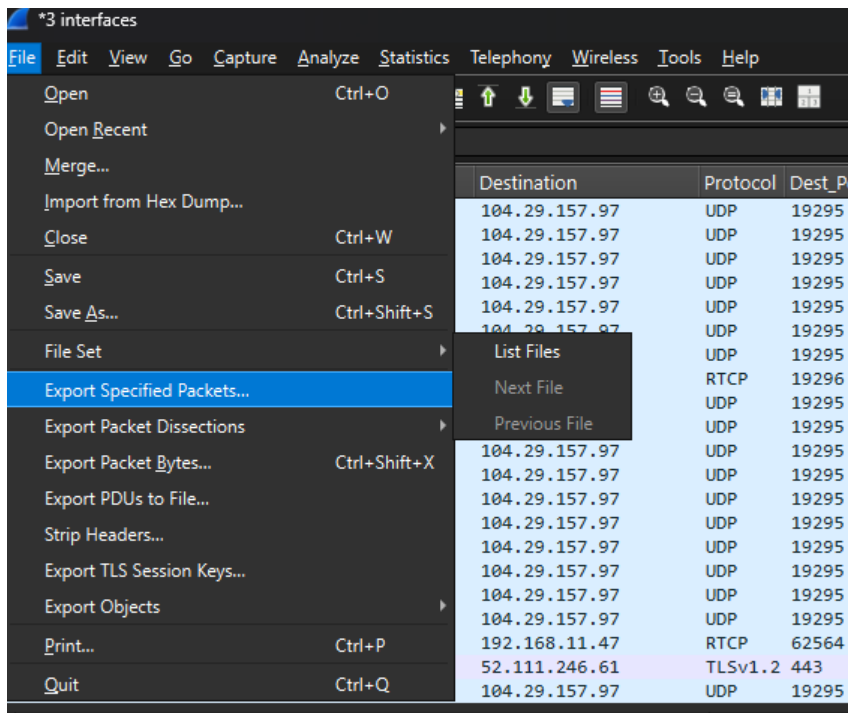
TCPDump Version:

- `tcpdump -i <interface> -C <size_in_MB> -W 2 -w /path/to/cap_file`

- Save a capture.



- Export specified packets to a new file.



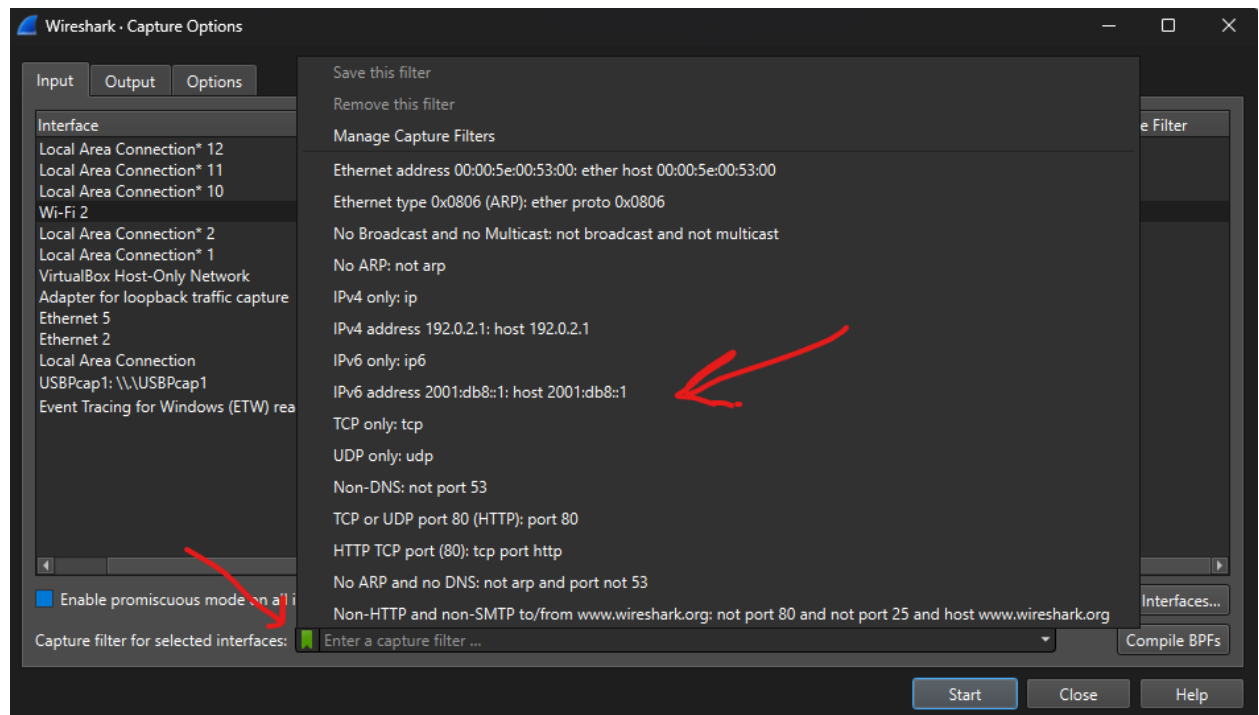
Export only works on filtered files, otherwise you export everything.

- Capture traffic using command line tools.
 Tcpcap in Linux - Probably not on your system for security reasons, ask admin.
 Tshark in Windows – Probably not in your base path, so move to correct directory.
- Describe the purpose of using promiscuous or monitor mode during a capture
 - **Promiscuous mode** captures all traffic on the *currently connected* network, even if addressed to other devices.
 - **Monitor mode** captures *all* wireless packets on a specific radio frequency, regardless of the network or access point, and is ideal for in-depth sniffing.

3.0 Filter traffic using capture and display filters. (12%)

-
- Compare and contrast Display Filters and Capture Filters.
-
- Capture filters** (set *before* capturing) limit which packets are saved to disk, reducing file size and system load.
- Display filters** (applied *after* capturing) hide uninteresting traffic in the current view without deleting it, allowing for detailed, deep-packet inspection.
-
- Implement a Capture Filter to capture only traffic from a single protocol, IP address,
-

MAC address, or port (range).



- Use multiple methods to create a Display Filter to isolate traffic for a single protocol or a property of a protocol. (manual entry, right click, drag/drop)
 1. Type it in at the top.
 2. You can right click on the actual row and column, to select filter.
 3. You can select the information in the bottom left corner and right click to apply the filter.
 - a. This can also be dragged right into the top display filter field and it will apply that filter to all display traffic.
- Use membership filters (tcp.port in {80,443})
https://www.wireshark.org/docs/wsug_html_chunked/ChWorkBuildDisplayFilterSection.html
- Use logical operators to connect multiple filters together.

Table 6.7. Display Filter Logical Operations

English	C-like	Description	Example
and	&&	Logical AND	<code>ip.src==10.0.0.5 and tcp.flags.fin</code>
or		Logical OR	<code>ip.src==10.0.0.5 or ip.src==192.1.1.1</code>
xor	^^	Logical XOR	<code>tr.dst[0:3] == 0.6.29 xor tr.src[0:3] == 0.6.29</code>
not	!	Logical NOT	<code>not ttlc</code>
[...]		Subsequence	See "Slice Operator" below.
in		Set Membership	<code>http.request.method in {"HEAD", "GET"}</code> . See "Membership Operator" below.

- Create a button for easy access to a Display Filter.

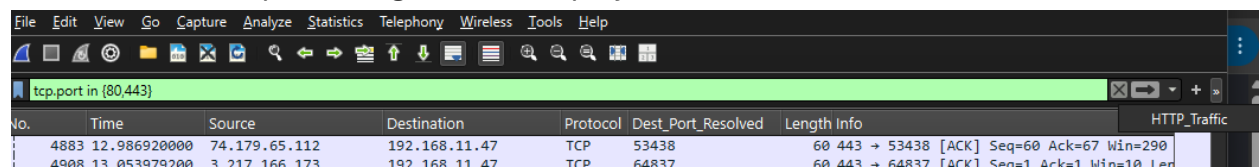
Type in your display filter

Click the button on the far right

Give it a name and a description

Hit "OK"

Button will show up to the right of the Display



- Identify situations where a Display Filter will show incomplete or excess results (i.e. filter for HTTP, but do not see the TCP handshake)

Incomplete

HTTP (as above)

RTP (will not show Session Initiation Protocol above it)

Too many && (and)s

Excessive

Too many || (or)s will expand too much

- Identify the behavior of using ! (not) in different parts of filter logic by explaining the implicit 'any' and 'all' qualifiers.

The meaning of != (all not equal) was changed in Wireshark 3.6. Before it used to mean "any not equal".

All Not (!=, all_ne): Matches if none of the fields in a packet meet the condition. Example: tcp.port != 80 (Shows packets where neither source nor destination port is 80).

Any Not (!~, any_ne): Matches if at least one field in a packet does not meet the condition. (Note: The all_ne is typically used for the logical "all not").

-
- Apply filters from Statistics > Conversations and Statistics > Endpoints.

Apply the filters in those locations and it will be in the display filter out in the normal panel.

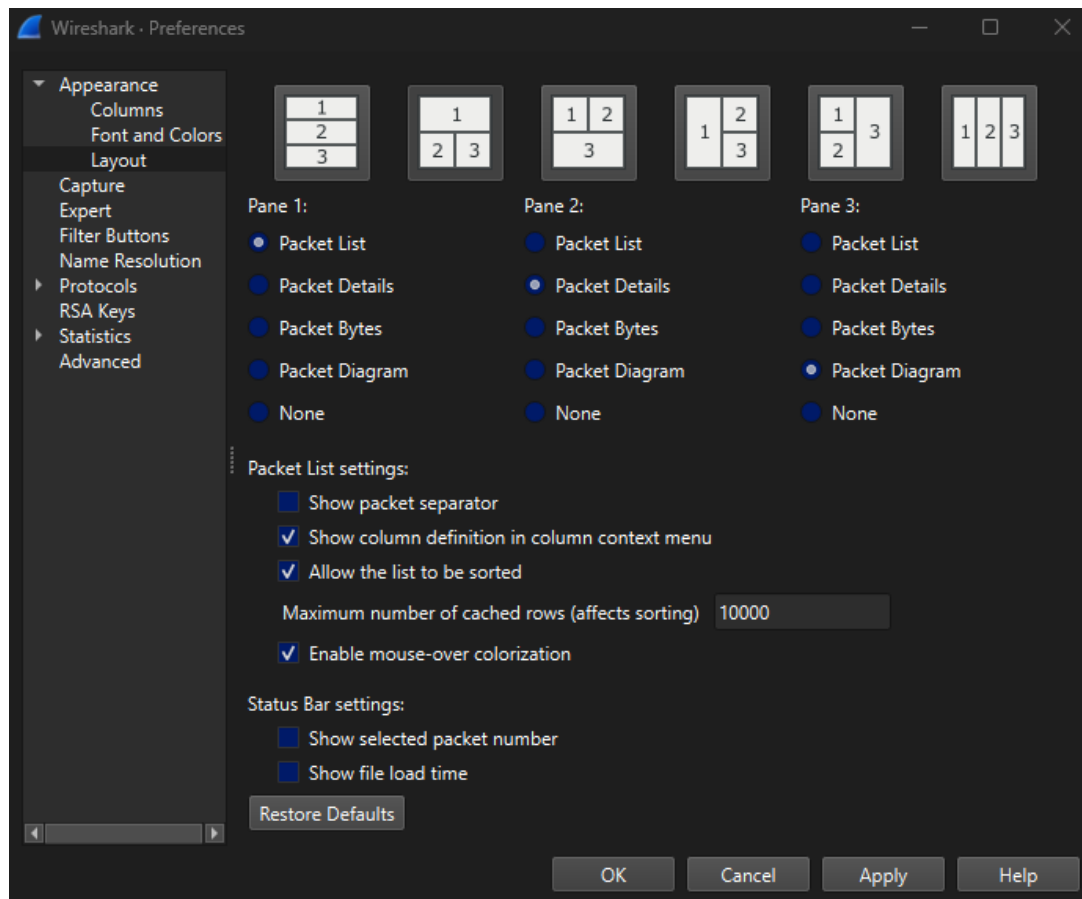
-
- Create a filter using generated fields in Wireshark.

Drag it up from the bottom left panel and put it in the display filter.

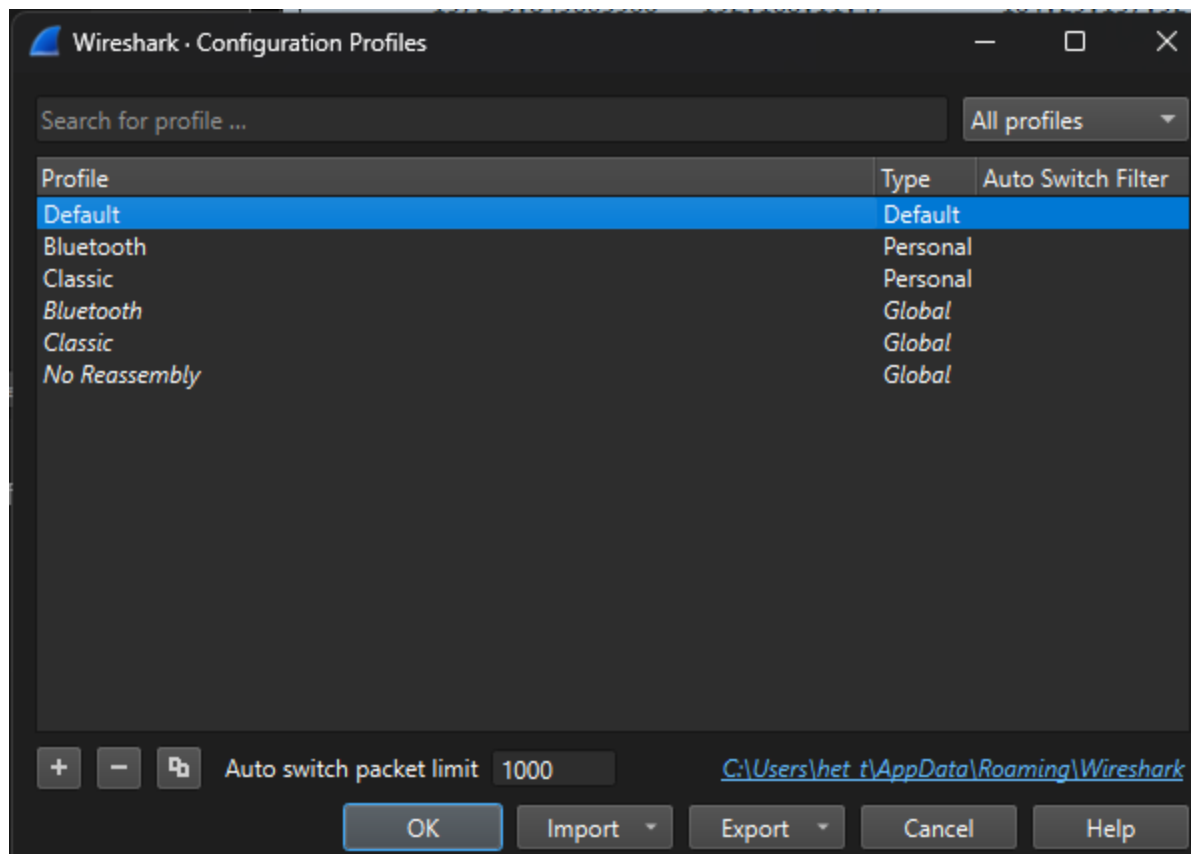
Notes from 9 May 2026

4.0 Configure, adapt, and use the Wireshark interface for different scenarios. (5%)

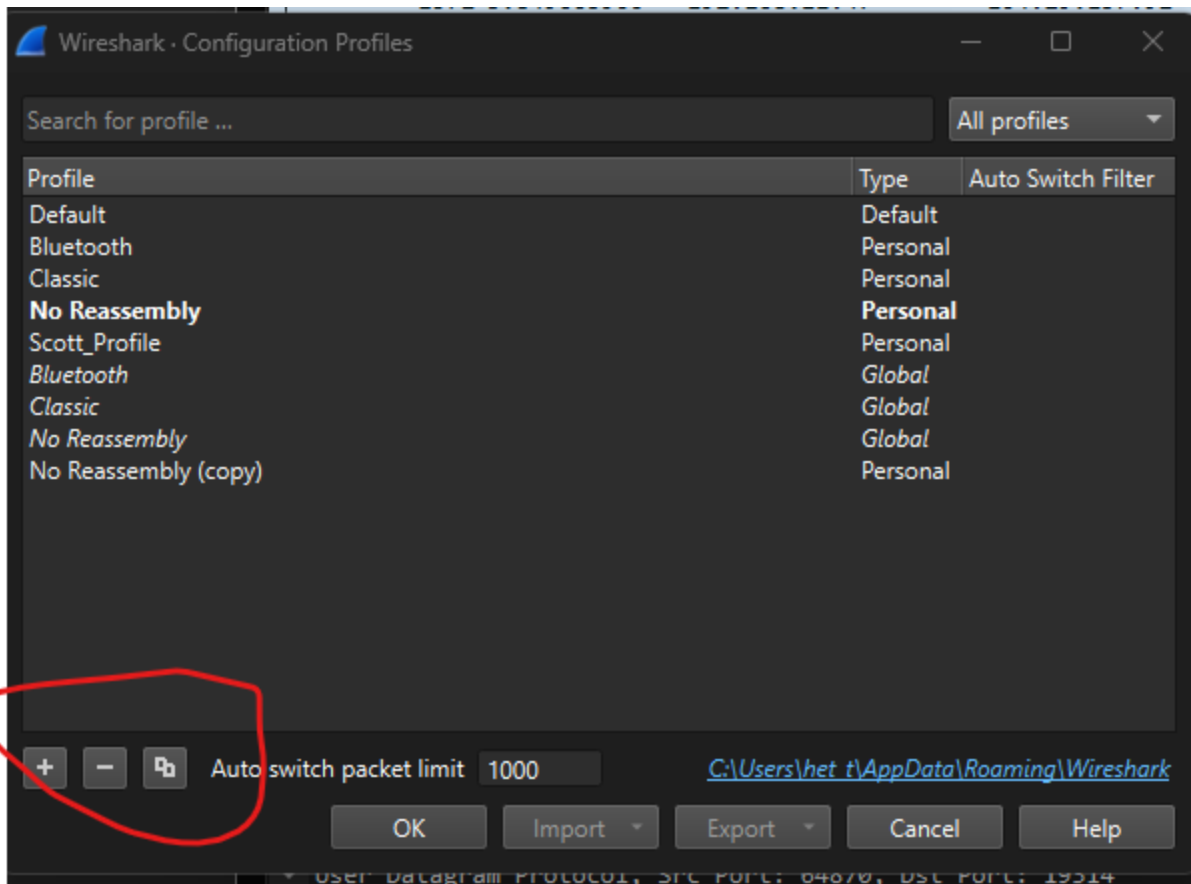
- Identify key components of the GUI (packet list, hex view, packet details, etc.).
 - Pick this in the Edit -> options -> Layout
- Modify panes with a different layout/features.
 - Same as above



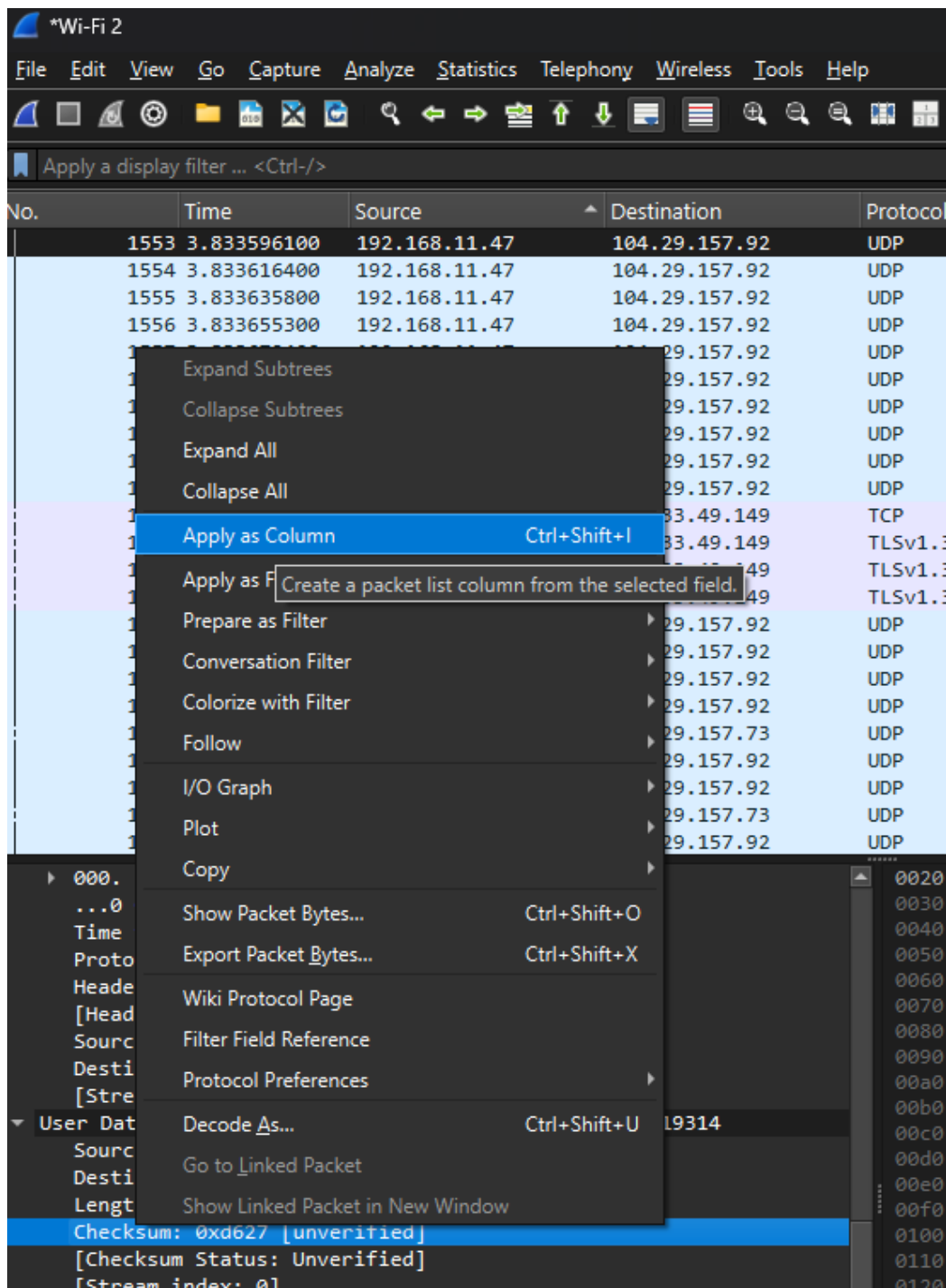
- Describe the value of using profiles.
 - Saves time giving you the views and window panes that you like to look at for certain traffic.
 - Profiles can be
 - Default
 - Personal
 - Global



- Create/modify/copy a profile.



- Describe the importance of columns in troubleshooting.
 1. I know the data.
 2. I can filter or sort the data.
- Use multiple methods to add a column.
 - Select from the packet view and then “apply as column”



Right click on a column header and click “Edit Column”

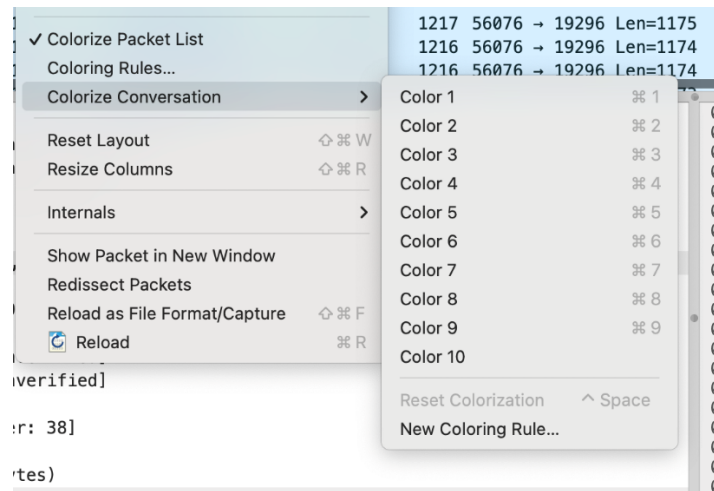
Apply a display filter ... <Ctrl-/>						
Title: Source		Type: Source address	Fields: Enter a cu...		Occurrence:	Display as: Strings
No.	Time	Source	Destination	Protocol	Length Info	
1542	3.833343700	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1543	3.833363900	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1544	3.833384400	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1545	3.833404200	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1546	3.833456000	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1547	3.833476800	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1548	3.833507200	192.168.11.47	104.29.157.73	UDP	89 60934 → 19296	Len=47
1549	3.833517000	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1550	3.833533000	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1551	3.833554200	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1552	3.833574700	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1553	3.833596100	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1554	3.833616400	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1555	3.833635800	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1556	3.833655300	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1557	3.833679400	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1558	3.833700200	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1559	3.833720400	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1560	3.833740400	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1561	3.833759700	192.168.11.47	104.29.157.92	UDP	1255 64870 → 19314	Len=1213
1563	3.836121000	192.168.11.47	104.29.157.92	UDP	203 64870 → 19314	Len=161
1568	3.841432000	192.168.11.47	34.233.49.149	TCP	54 57224 → 443 [ACK] Seq=453 Ack=4385 Win=65280 Len=0	

- Use coloring rules to highlight packets.
 - There are defaults (below)
 - There are also custom rules you can apply to conversations.

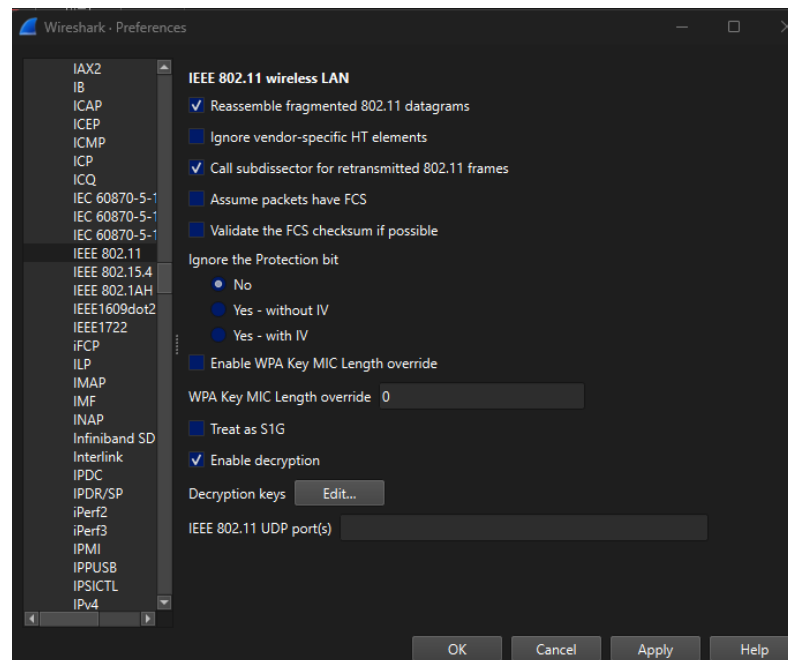
Wireshark · Coloring Rules SmD	
Name	Filter
☒ Bad TCP	tcp.analysis.flags && !tcp.analysis.window_update && !tcp.analysis.keep_alive && !tcp.analysis.ke
☒ TCP Syn	tcp.flags.syn==1
☒ HSRP State Change	hsrp.state != 8 && hsrp.state != 16
☒ Spanning Tree Topology Change	stp.type == 0x80
☒ OSPF State Change	ospf.msg != 1
☒ ICMP errors	icmp.type in { 3..5, 11 } icmpv6.type in { 1..4 }
☒ ARP	arp
☒ ICMP	icmp icmpv6
☒ TCP RST	tcp.flags.reset eq 1
☒ SCTP ABORT	sctp.chunk_type eq ABORT
☒ IPv4 TTL low or unexpected	(ip.dst != 224.0.0.0/4 && ip.ttl < 5 && !(pim ospf eigrp bgp tcp.port == 179)) (ip.dst == 224
☒ IPv6 hop limit low or unexpected	(ipv6.dst != ff00::/8 && ipv6.hlim < 5 && !(ospf bgp tcp.port == 179)) (ipv6.dst == ff00::/8 &&
☒ Checksum Errors	eth.fcs.status == "Bad" ip.checksum.status == "Bad" tcp.checksum.status == "Bad" udp.checksu
☒ SMB	smb nbss nbns netbios
☒ HTTP	http tcp.port == 80 http2
☒ DCERPC	dcerpc
☒ Routing	hsrp eigrp ospf bgp cdp vrrp carp gvrp igmp ismp
☒ TCP SYN/FIN	tcp.flags & 0x02 tcp.flags.fin == 1
☒ TCP	tcp
☒ UDP	udp
☒ Broadcast	eth[0] & 1
☒ System Event	systemd_journal sysdig

- Use the minimap (colored sidebar) to quickly locate packets of interest.
 - Right side shows a complete view of the entire capture, to jump around.
- Use the 'Colorize Conversation' feature.

- Right click on a packet
- Or view menu to colorize a conversation.
- In Mac (Apple)



- Understand the importance of protocol preferences in your analysis.
 - Edit -> Preferences -> Protocols
 - Then you can set defaults for different protocols based on how they are set in your environment.



- Use the mark/unmark packet feature.
 - Time shifts to 0.00000 related to each mark of time shift.

Notes from 16 May 2026

5.0 Identify and explain common network protocols dissected by Wireshark. (43%)

(note: TCP is 17% of exam and is scored separately. Non-TCP protocols are 27% of exam)

5.1 ETHERNET

- Identify Fields of Ethernet frame.
 - Destination MAC
 - 48 bytes long
 - First 24 are the vendor
 - Last 24 are unique to the device
 - It's all made up in virtual space, but the rule is, they must be locally unique as they are used in delivery of Frames to endpoints.
 - Source MAC
 - Type
 - Data
 - CRC (Trailer)

Reference: https://en.wikipedia.org/wiki/Ethernet_frame

- Describe minimum and maximum frame sizes.
 - Read these as MTU (Maximum Transmission Units)
 - Typically 1500 Bytes
 - There are Jumbo Frames 9000+ Bytes
- Explain why the CRC is missing from Ethernet frame in Wireshark.
 - Stripped off by the system (Hardware Stripping)
- Identify common Ethertypes (IPv4, IPv6, ARP).
 - ARP
 - Has Padding applied
 - IPv4
 - Dest, Src, and then type (aware of IPv4 traffic) before the data.
 -
- Distinguish between Unicast, Broadcast, and Multicast MAC addresses.
 - Unicast (directly to a destination host)
 - Broadcast (To all hosts FF:FF:FF:FF:FF:FF)
 - Multicast (Single group of devices, by the first half of the MAC)

- https://en.wikipedia.org/wiki/Multicast_address
- Describe how the frame header is modified to accommodate VLAN tags.
 - 802.1Q
 - There is a field added that really only matters to switches. Switches do not allow frames tagged with 802.1Q to egress a port that does not have the same vlan tag.

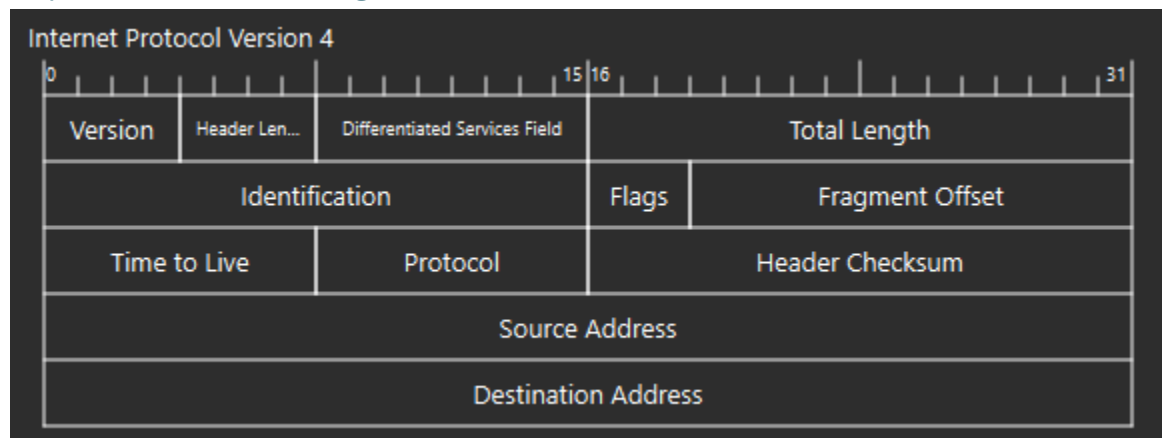
5.2 ARP

- Describe the purpose of the ARP protocol.
 - Mapping IP (Logical – Hierarchical addresses) to MAC (Physical – non-hierarchical)
 -
- Identify and explain the purpose of different types of ARP packets.
 - Requests (Who has?) – Sent via MAC broadcast
 - Replies (I have) – Sent via MAC unicast
- Create filters for different types of ARP traffic.
 - Requests look like this
 - `arp.opcode == 1`
 - `(eth.dst == ff:ff:ff:ff:ff:ff) && (_ws.col.protocol == ARP)`
 - Replies look like this
 - `!(eth.dst == ff:ff:ff:ff:ff:ff) && (_ws.col.protocol == ARP)`
 - `arp.opcode == 2`
- Describe the difference between a broadcast ARP and a unicast ARP.
 - ARP broadcast is always the request. The Unicast is a system responding directly back what IP address it has.

5.3 IPv4

- Describe common features of the IP protocol.
 - Hierarchical
 - There are two versions.
 - IPv4
 - Flags
 - Tell us if we can fragment
 - Tell us if more fragments are coming (offset)
 - TTL (Time to Live) is decremented at each hop of a router.
 - Protocol – Tells us the upper layer protocol encapsulated (TCP, ICMP, UDP)

- Address Fields
 - 32 bit address
 - Source
 - Destination
- IPv6
 - 128 bit address
- Describe header values of the IP protocol (TTL, Fragmentation, Packet Length, Protocol)
 - Know the fields below at a high level.
 - Very important to review these in light of the RFC Protocol
<https://datatracker.ietf.org/doc/html/rfc791>



ID, IP ID).

- Describe public, private, multicast and APIPA IP address ranges.
 - Public
 - All addresses that are routable on the internet
 - Private (These will never leave your network, nor traverse the internet)
 - 10.0.0.0 – 10.255.255.255
 - 172.16.0.0 – 172.31.255.255
 - 192.168.0.0 – 192.168.255.255
 - 127.0.0.0 – 127.255.255.255 (Loopback) (Home)
 - Multicast
 - 224.0.0.0 to 239.255.255.255
 - APIPA Ranges
 - 169.254.0.1 to 169.254.255.254 – These ranges are assigned to a device that gets no DHCP response (internal talking only).

- Describe how NAT works and why this should be considered when capturing and analyzing network traffic.
 - NAT – Network Address Translation – You must capture traffic before NAT occurs, or the entire internal network is hidden to you.
 - One to one – Mapping (requires the same internal as external IP addresses. This is typically not technically feasible)
 - One to many – Mapping
 - PAT – Port Address Translation – This is a one external to many internal, but using port numbers to map traffic
 - One to many – But using port numbers.
- Identify and explain the purpose of the IP TTL field.
 - Counts down to zero at every hop of a network device. The device decrements this field as it passes through to the other interface.
- Predict most likely distance, in hops, from the capturing device.
 - UDP Typically will be 128 – number of hops = Current value of the field.
 - TCP Typically will be 64 – number of hops = Current value of the field.
- Describe different IP identification strategies and how to use them in troubleshooting.
 - You can know what is internal to your network by looking at your CIDR /24 (or other mask).

5.4 ICMPv4

- Identify ICMP message types and their purpose.
 - PING – Packet Interworking Groper
 - ICMP Echo Request
 - ICMP Echo Reply

Study these: https://en.wikipedia.org/wiki/Internet_Control_Message_Protocol

- Identify the IP packet which triggered an ICMP error message or reply.
 - There is an arrow indicator -> (Request) and <- (Reply) that you have sent and replied in Wireshark when you are focused (or selected) a packet

→	1978	3.851605900	23:18:37.7095578Z	192.168.11.47	104.29.152.118	UDP	205	59118 → 19331	Len=163
	1979	3.852738100	23:18:37.7106900Z	192.168.11.47	192.168.200.51	ICMP	74	Echo (ping) request	id=0x0001
	1980	3.855803900	23:18:37.7137558Z	192.168.11.1	192.168.11.47	ICMP	106	Redirect	(Redirect
←	1981	3.855923000	23:18:37.7138749Z	192.168.200.51	192.168.11.47	ICMP	74	Echo (ping) reply	id=0x0001
	1982	3.861297700	23:18:37.7192496Z	192.168.11.47	104.29.152.25	UDP	261	60128 → 19300	Len=219

5.5 IPv6

- Identify types of IPv6 addresses (Link Local, Global Unicast, Multicast).

5.6 ICMPv6

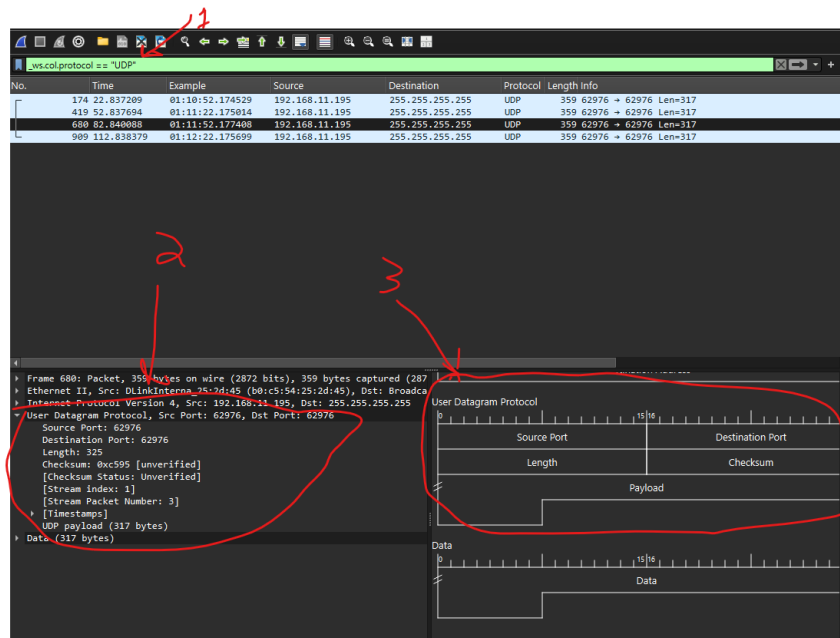
- Identify and explain components of neighbor/router discovery protocol.
- Identify and explain purpose of Neighbor Advertisements/Solicitations.

Notes from 23 May 2026

5.0 Identify and explain common network protocols dissected by Wireshark. (continued)

5.7 UDP

- Identify UDP traffic.
 1. UDP in the protocol field – Can filter by that protocol field
 2. UDP shown in frame
 3. UDP Header shown in header section



- Identify higher layer protocols which use UDP.
 - SIP – Session Initiation Protocol – Starts VOIP calls
 - RTP – Real-time Transport Protocol – unencrypted VOIP call data
 - SRTP – Secure Real-time Transport Protocol – encrypted VOIP call data

- RTCP – Real-time Transport Control Protocol – Control messaging for RTP
- DNS – Domain Name Service – IP to Common name mapping
- SSDP – Simple Service Discovery Protocol -
- Describe the UDP stream id and conversation timestamps.
 - Stream ID is added by Wireshark. It is not part of the information in UDP.
- Describe why UDP is used in multicast or broadcast IP traffic.
 - It's faster, and doesn't require the establishment or teardown of a session.
 - There are no "acknowledgements" before next packets. Just shotgun the data across the network.

5.8 DHCPv4

- Identify and describe the 4 phases (DORA).
 - Discover (DHCPDISCOVER)
 - Offer (DHCPOFFER)
 - Request (DHCPREQUEST)
 - Acknowledge (DHCPACK)
- Describe the different purposes of a 'DHCP Request' message.
- Identify DHCP options and parameters (router, dns, subnetmask, custom options).
- Describe APIPA and how it relates to DHCP.
 - APIPA is a fallback if a system fails to get a DHCP lease. Your system configures itself with a 169.x.x.x address.
 - If we see an APIPA address, we should start to troubleshoot why our DHCP is not responding on the network, or why this system is unable to talk to it.

5.9 DNS

- Identify DNS requests and replies.

See queries and responses.

_ws.col.protocol == "MDNS"						
No.	Time	Example	Source	Destination	Protocol	Length Info
7	0.636810	01:10:29.974130	192.168.11.195	224.0.0.251	MDNS	111 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
30	3.646384	01:10:32.983704	192.168.11.195	224.0.0.251	MDNS	111 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
190	25.027375	01:10:54.364695	192.168.11.195	224.0.0.251	MDNS	77 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
191	25.057207	01:10:54.394527	192.168.11.195	224.0.0.251	MDNS	361 Standard query response 0x0000 PTR D-Link HNAP Se
201	26.040295	01:10:55.377615	192.168.11.195	224.0.0.251	MDNS	111 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
223	29.056650	01:10:58.393970	192.168.11.195	224.0.0.251	MDNS	111 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
386	50.468626	01:11:19.805946	192.168.11.195	224.0.0.251	MDNS	77 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
391	50.549620	01:11:19.886940	192.168.11.195	224.0.0.251	MDNS	361 Standard query response 0x0000 PTR D-Link HNAP Se
394	51.476861	01:11:20.814181	192.168.11.195	224.0.0.251	MDNS	111 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
442	54.502557	01:11:23.839877	192.168.11.195	224.0.0.251	MDNS	111 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
622	75.918021	01:11:45.255341	192.168.11.195	224.0.0.251	MDNS	77 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
623	75.967814	01:11:45.305134	192.168.11.195	224.0.0.251	MDNS	361 Standard query response 0x0000 PTR D-Link HNAP Se
631	76.927818	01:11:46.265138	192.168.11.195	224.0.0.251	MDNS	111 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
657	79.937020	01:11:49.274340	192.168.11.195	224.0.0.251	MDNS	111 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
818	101.297496	01:12:10.634816	192.168.11.195	224.0.0.251	MDNS	77 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
820	101.417550	01:12:10.754870	192.168.11.195	224.0.0.251	MDNS	361 Standard query response 0x0000 PTR D-Link HNAP Se
830	102.312292	01:12:11.649612	192.168.11.195	224.0.0.251	MDNS	111 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"
856	105.327403	01:12:14.664723	192.168.11.195	224.0.0.251	MDNS	111 Standard query 0x0000 PTR _dnhap._tcp.local, "QM"

- Use DNS information to identify relevant traffic.
- Distinguish between different DNS record types.

5.10 TCP (17%)

- Describe the components of a 3 way handshake.
 - SYN – Initial connection attempt
 - SYN ACK – Acknowledgement from server
 - ACK – Ack from initial sender
- Describe different methods TCP session are torn down.
 - FIN - One side chooses to stop transmission
 - FIN ACK – Other side acknowledges
- Explain how RTT is measured.
 - It is the time it takes a packet to travel from the sender to receiver and back.
 - It is calculated from the initial syn and syn/ack because there is no additional buffering from protocols.
 - It is a fixed reference for the rest of this session.
- Describe and calculate Maximum Segment Size(MSS).
 - Maximum size that a single TCP segment can carry as its payload not including the TCP or IP headers
 - For Ethernet (MTU 1500) the default MSS is 1460 Bytes
$$\text{MSS} = \text{MTU} - (\text{TCP Header} + \text{IP Header})$$
$$1460 = 1500 - (20 + 20)$$
<https://www.rfc-editor.org/rfc/rfc6691.html>
 - Negotiated at the three way handshake, devices both inform their maximum capabilities and the lower of the two is selected.
 - MSS Clamping sometimes occurs within VPN/Tunneling because of an overhead. This is so that tunnel encapsulation does not cause packet drops.
- Describe how TCP flags are used to establish and tear down a TCP session.

The image shows a Wireshark packet capture of a TCP ACK packet. The packet list pane shows a packet with sequence number 1091 and acknowledgment number 1334. The packet details pane shows the TCP header with sequence number 1091, acknowledgment number 1334, and window size 250. The packet bytes pane shows the raw data of the packet.

- Explain how sequence and acknowledgment numbers are used to maintain reliability.
- Explain TCP options and their purpose.(EOL, NOP, MSS, SACK, DSACK, Window Scaling)

A breakdown of how each component functions and its specific purpose:

1. MSS (Maximum Segment Size)

- **Purpose:** Dictates the maximum amount of payload data (in bytes) a device is willing to receive in a single packet.
- **Reason for Use:** Prevents IP fragmentation. If the MSS is perfectly aligned with the MTU (Maximum Transmission Unit) of the network (typically 1460 bytes for standard Ethernet with a 20-byte TCP header and 20-byte IP header), the packets can route without being broken into smaller, inefficient fragments. [1, 2, 3, 4]

2. NOP (No-Operation)

- **Purpose:** A single-byte padding mechanism (01 hex) that aligns subsequent TCP options to exact 4-byte boundaries.
- **Reason for Use:** TCP options must end on a 32-bit boundary. Using a NOP ensures that specific options (like Window Scale or Timestamps) are properly structured in memory and easily read by system hardware and drivers. [1, 2]

3. EOL (End of Option List)

- **Purpose:** A single-byte flag (00 hex) that explicitly tells the receiving stack that no more TCP options follow in the current header.
- **Reason for Use:** It pads the remaining header space up to the 32-bit boundary, ensuring the receiver does not read garbage data, while also signaling the immediate conclusion of the option field. [1, 2, 3]

4. SACK (Selective Acknowledgment)

- **Purpose:** An extension that alters standard acknowledgment behavior. Instead of only acknowledging the last sequential packet received (e.g., acknowledging up to sequence 100, when 101-200 dropped), SACK explicitly informs the sender of exactly which packet blocks were successfully received. [1]
- **Reason for Use:** It prevents the sender from retransmitting perfectly good packets. If packets 10 and 12 successfully arrive, but packet 11 is lost, SACK allows the sender to resend *only* packet 11, drastically boosting efficiency during network packet loss. [1]

5. DSACK (Duplicate Selective Acknowledgment)

- **Purpose:** An extension of SACK that allows the receiver to report duplicate packets.
- **Reason for Use:** Used for diagnostics and performance tuning. It helps the TCP sender figure out whether a retransmission was actually necessary (e.g., if a packet was just delayed, not lost) or if its own acknowledgment (ACK) packet got lost in transit. [1, 2, 3, 4]

6. Windows Scaling (TCP Window Scale)

- **Purpose:** Multiplies the TCP receive window size. The standard TCP header allocates a 16-bit field for the window size, artificially capping it at 65,535 bytes. [1, 2]
- **Reason for Use:** In high-speed, high-latency networks (Long Fat Networks - LFN), a 64 KB window causes the sender to constantly pause and wait for acknowledgments. Window scaling allows the window to open to gigabyte levels, allowing continuous, high-speed data streams without artificial pacing constraints.

- Describe the purpose of a DUP ACK.
 - The purpose of a duplicate acknowledgment (Dup ACK) in TCP is to notify the sender that a packet was lost or delivered out-of-order, and to prompt the sender to resend the missing data before waiting for a timeout.
- Identify the byte range of missing segments based on a DUP ACK with SACK or DSACK.

No.	Time	Example	Source	Destination	Protocol	Length	Info
86	0.243129	01:08:08.189896	189.244.116.39	192.168.11.246	TCP	66	[TCP Dup ACK 84#1] 47217 → 22 [ACK]

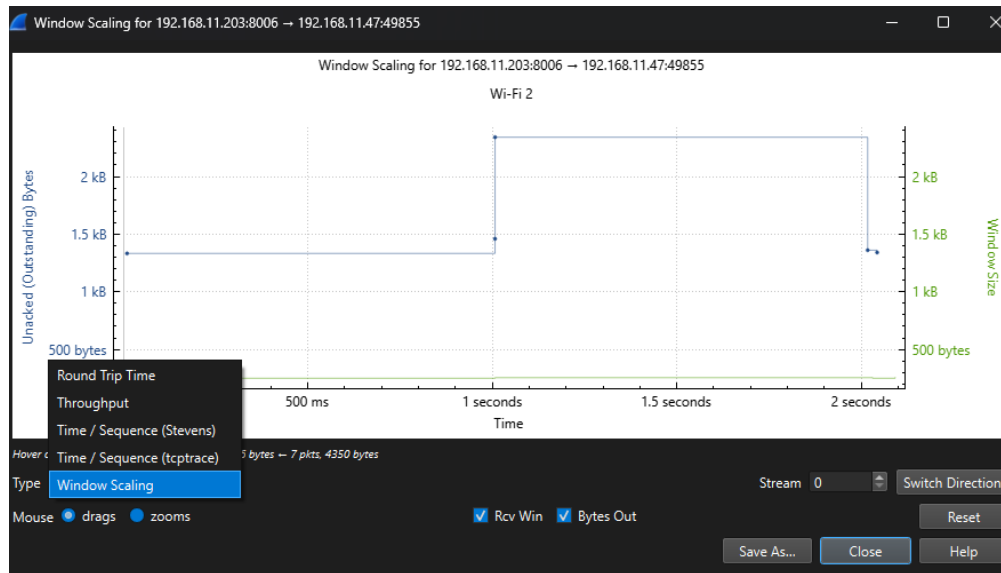
- The acknowledgement number is what the receiver is asking for

The image shows a Wireshark capture of a duplicate ACK packet. The packet list pane at the top shows a packet from 189.244.116.39 to 192.168.11.246, identified as a duplicate ACK. The packet details pane shows the following information:

- Transmission Control Protocol, Src Port: 47217, Dst Port: 22, Seq: 201
- Source Port: 47217
- Destination Port: 22
- [Stream index: 1]
- [Stream Packet Number: 12]
- [Conversation completeness: Incomplete (28)]
- [TCP Segment Len: 0]
- Sequence Number: 201 (relative sequence number)
- Sequence Number (raw): 3702382055
- [next Sequence Number: 201 (relative sequence number)]
- Acknowledgment Number: 537 (relative ack number)
- Acknowledgment number (raw): 577401053
- 1000 = Header Length: 32 bytes (8)
- Flags: 0x010 (ACK)

The packet bytes pane on the right shows the Ethernet II frame structure with Destination and Source MAC addresses, and the Internet Protocol Version 4 header.

- Describe what each line represents in a TCP Stream Graph.
 - Statistics -> TCP Stream Graphs



- Describe the purpose of TCP reassembly in Wireshark.
 - The process of reconstructing segmented application data (like HTTP or DNS) that spans multiple packets. It stitches the fragmented payload together so you can analyze the complete transaction in a single, readable view.
- Describe the TCP stream id and conversation timestamps.
 - TCP Stream ID is metadata, and is generated by wireshark. a unique, application-generated number used to identify an entire conversation between two endpoints, starting from zero.
 - Conversation timestamps – These are also generated in wireshark and are relative to the start and also time since the previous frame in the same stream.

Notes from 30 May 2026

6.0 Use Wireshark to troubleshoot common issues with protocols

listed above. (20%)

- Determine network topology only using information in a packet capture.
- Perform TCP sequence and acknowledgment number analysis.
- Distinguish server performance from slow transfer times (for instance in HTTP).

- Identify the effect of high RTT on request/response protocols. (For example: HTTP, SMB, SQL)
- Identify the effect of a low window size in combination with high RTT.
- Given a capture, identify potential network communication issues using information from ARP, DHCP, and ICMP.

Additional Study Materials